FOOD ORDERING SYSTEM

ABSTRACT:

The Food Ordering System is a console-based Java application designed to let a customer browse a restaurant menu, add food items to a cart, and generate a bill automatically. Unlike manual order taking, which is slow and prone to calculation mistakes, this system uses simple array-based data structures to store food names, prices, and ordered quantities, and performs all billing calculations instantly and accurately.

Each food item is identified by its index position in three parallel arrays: Food Name, Price, and Quantity. The menu choice entered by the user maps directly to an array index, allowing every item to be accessed in constant time. The menu-driven loop lets the user add items repeatedly, with quantities accumulating in the cart, and then view an itemized bill with the total amount at any time.

Core operations implemented include: Display Menu — for showing available food items and prices; Add to Cart — for selecting an item and quantity with input validation; Generate Bill — for calculating the amount per item and the grand total; Exit — for ending the program gracefully. The system is particularly suitable as an academic or foundational project demonstrating how arrays, loops, conditional statements, and the Scanner class can solve a real-world ordering problem, and serves as a stepping stone toward more complex database-backed ordering solutions.

Keywords: Java, Food Ordering, Arrays, Scanner, Menu-Driven Program, Loops, Billing, Input Validation.

INTRODUCTION:

In today's world, ordering food quickly and accurately is essential for restaurants and canteens. Manual methods of taking orders and preparing bills are slow, error-prone, and inefficient. A Food Ordering System solves this by automating the process of selecting items, recording quantities, and calculating the final bill.

This project implements a Food Ordering System in Java using arrays, where the item name, price, and ordered quantity of each dish are stored at the same index of three arrays. The program runs in a continuous menu loop, accepts user input through the Scanner class, validates the entered choice and quantity, and generates a neat bill on demand. The goal of this project is to demonstrate how basic programming constructs can be applied to a real-world problem in an efficient and organized way.

ALGORITHM:

Step 1: Start

Step 2: Define Data Structures

Each food item is stored at the same index in three arrays:

food[ ] : Pizza, Burger, Biryani, Sandwich, Paneer Roll

price[ ] : 150, 80, 180, 60, 100

quantity[ ] : size 5, all values initially 0

Step 3: Initialize

Create Scanner object sc to read input

Create array quantity[5] (all values = 0)

Step 4: Display Menu (Repeat until exit)

1. Pizza 2. Burger 3. Biryani 4. Sandwich 5. Paneer Roll

6. Generate Bill

7. Exit

Step 5: Perform Operations

5a. Add to Cart

Input choice (1 to 5)

Input quantity q

If q > 0

quantity[choice-1] = quantity[choice-1] + q

Print "<food> added to cart successfully!"

Else

Print "Enter a valid quantity!"

5b. Generate Bill

Set total = 0

For i = 0 to 4

If quantity[i] > 0

amount = quantity[i] * price[i]

Print food[i], quantity[i], price[i], amount

total = total + amount

If total == 0

Print "Your cart is empty!"

Else

Print "Total Amount" and "Order placed successfully!"

5c. Exit

Print "Thank you for visiting!"

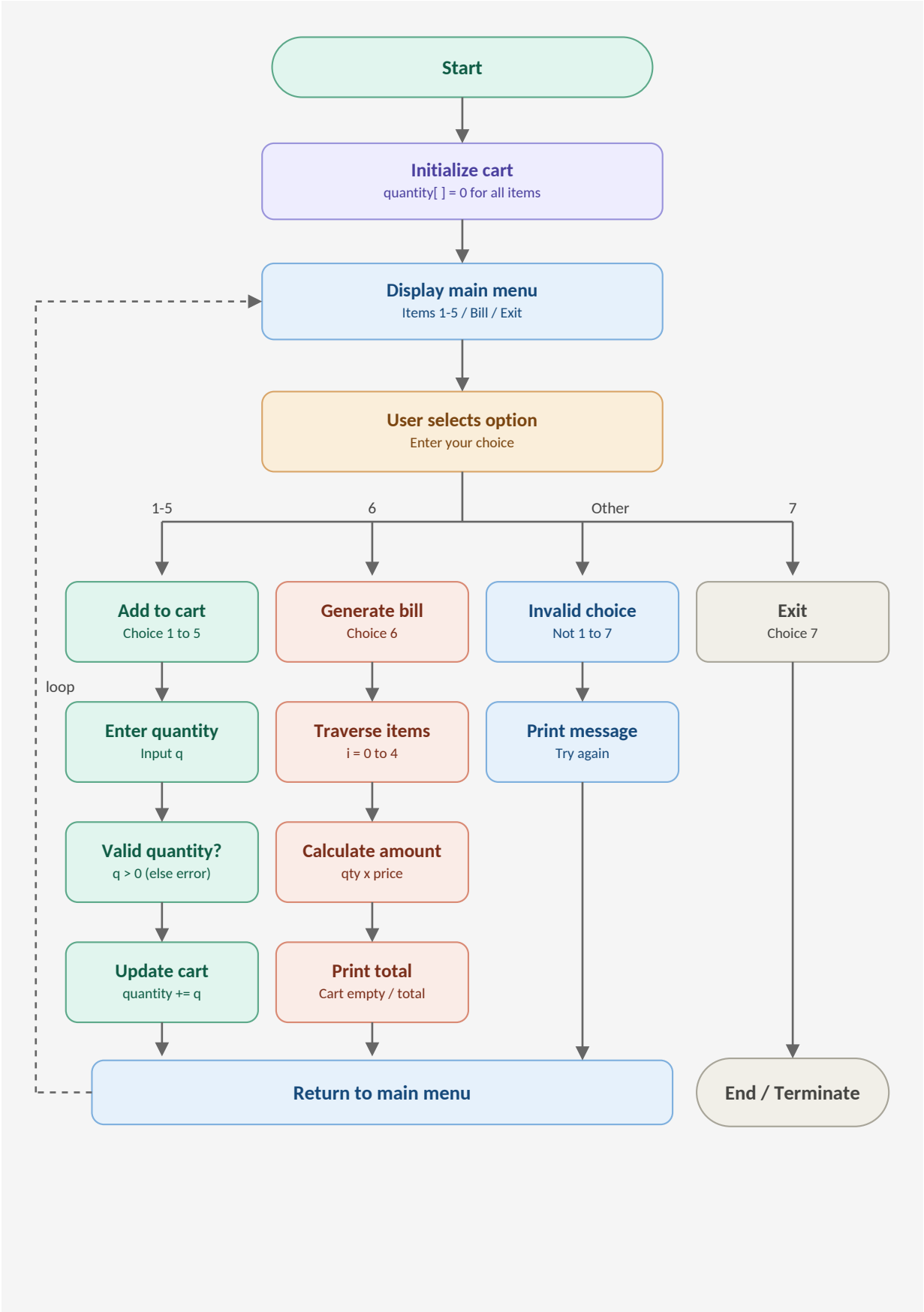
Break the loop and stop program

5d. Invalid Choice

If choice is not 1 to 7 -> print "Invalid choice! Try again."

Step 6: Close Scanner and Stop

FLOW CHART:


Start
Initialize cart
quantity[ ] = 0 for all items
Display main menu
Items 1-5 / Bill / Exit
User selects option
Enter your choice
1-5
6
Other
7
Add to cart
Choice 1 to 5
Generate bill
Choice 6
Invalid choice
Not 1 to 7
Exit
Choice 7
loop
Enter quantity
Input q
Traverse items
i = 0 to 4
Print message
Try again
Valid quantity?
q > 0 (else error)
Calculate amount
qty x price
Update cart
quantity += q
Print total
Cart empty / total
Return to main menu
End / Terminate

METHODOLOGY:

The Food Ordering System project is designed to efficiently manage the process of taking food orders and preparing bills. The core ideology of this system is to ensure accuracy, automation, and speed in order handling. The system follows a menu-driven approach where all food items and their prices are stored in one place, making it easy to display, select, and manage. It reduces manual work by automating key processes such as cart updates, amount calculation, and bill generation. Another important aspect of the ideology is keeping the interaction simple, so that any user can place an order without training.

The program stores food names, prices, and quantities in parallel arrays, so that the choice entered by the user directly selects the correct item. A while loop keeps displaying the menu until the user chooses to exit, and decision statements (if / else if / else) handle each option. Input validation ensures that only valid menu choices and positive quantities are accepted, and repeated orders of the same item are added to the existing quantity. When the bill is requested, a for loop calculates the amount of every ordered item, adds it to the running total, and prints an itemized bill. Overall, the ideology of the Food Ordering System is to create a simple, efficient, and scalable solution that minimizes errors, saves time, and improves the customer experience.

RESULT:

Food Ordering System - Console

```
===== FOOD ORDERING SYSTEM =====
1. Pizza      - Rs.150
2. Burger     - Rs.80
3. Biryani    - Rs.180
4. Sandwich   - Rs.60
5. Paneer Roll- Rs.100
6. Generate Bill
7. Exit
Enter your choice: 1
Enter quantity: 2
Pizza added to cart successfully!

===== FOOD ORDERING SYSTEM =====
1. Pizza      - Rs.150
2. Burger     - Rs.80
3. Biryani    - Rs.180
4. Sandwich   - Rs.60
5. Paneer Roll- Rs.100
6. Generate Bill
7. Exit
Enter your choice: 2
Enter quantity: 3
Burger added to cart successfully!

===== FOOD ORDERING SYSTEM =====
1. Pizza      - Rs.150
2. Burger     - Rs.80
3. Biryani    - Rs.180
4. Sandwich   - Rs.60
5. Paneer Roll- Rs.100
6. Generate Bill
7. Exit
Enter your choice: 6

========== YOUR BILL ==========
Food            Qty     Price   Amount
Pizza   2       150     300
Burger  3       80      240
------------------------------
Total Amount: Rs.540
Order placed successfully!
==============================

===== FOOD ORDERING SYSTEM =====
1. Pizza      - Rs.150
2. Burger     - Rs.80
3. Biryani    - Rs.180
4. Sandwich   - Rs.60
5. Paneer Roll- Rs.100
6. Generate Bill
7. Exit
Enter your choice: 7
Thank you for visiting!

--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

CONCLUSION:

This project successfully demonstrates the design and implementation of a Food Ordering System using Java. The system effectively addresses the core challenges of order taking by providing a simple, fast, and structured approach to selecting food items, storing quantities, and generating bills.

Through this project, the practical application of arrays, loops, and conditional statements in solving real-world problems has been clearly established. The use of parallel arrays allowed each item's name, price, and quantity to be accessed directly through its index, while the menu-driven while loop allowed the user to place multiple orders without restarting the program. Input validation prevented invalid choices and negative or zero quantities from corrupting the cart.

The key operations implemented — displaying the menu, adding items to the cart, calculating amounts, and generating the bill — proved that fundamental programming concepts can be applied effectively to build functional and meaningful software solutions. The system eliminates manual calculation errors and provides a fast and organized way to prepare an order bill.

From an academic perspective, this project strengthened the understanding of array handling, control flow, user input using the Scanner class, and algorithm design through hands-on implementation. It bridges the gap between theoretical programming concepts and their practical, real-world applications.

In conclusion, the Food Ordering System serves as a strong foundation for more advanced ordering solutions. With further enhancements such as file handling for data persistence, graphical user interfaces, database integration, and online payment support, this system can be scaled into a fully functional restaurant ordering application.

FUTURE WORK:

Although the current system fulfills its basic objectives, several enhancements can be made in the future:

- Database Integration — Store the menu, customers, and orders in a database like MySQL to enable permanent data storage and faster retrieval.
- File Handling — Add file read/write functionality to save order history and bills even after the program closes.
- Graphical User Interface (GUI) — Develop a user-friendly GUI using tools like Java Swing or JavaFX instead of a console-based interface.
- Dynamic Menu Management — Replace fixed arrays with ArrayList or a linked list so that admins can add, update, or remove food items at runtime.
- Remove / Edit Cart Items — Allow customers to remove items or change quantities before generating the final bill.
- Discounts & Taxes — Add GST calculation, coupon codes, and discount offers to the final bill.
- Multi-user Support — Add login and role-based access control for Customer, Staff, and Admin users.
- Online Payment Integration — Enable UPI, card, or wallet payments directly from the application.
- Cloud-based System — Host the system online (web or mobile app) to allow orders from multiple locations in real time.